

COURSE OBJECTIVES:

- To familiarize with C programming constructs.
- To develop programs in C using basic constructs.
- To develop programs in C using arrays.
- To develop applications in C using strings, pointers, functions.
- To develop applications in C Using structures.
- To develop applications in C using file processing.

LIST OF EXPERIMENTS:

Note:The lab instructor is expected to design problems based on the topics listed. The Examination shall not be restricted to the sample experiments designed.

1. I/O statements, operators, expressions
2. decision-making constructs: if-else, goto, switch-case, break-continue
3. Loops: for, while, do-while
4. Arrays: 1D and 2D, Multidimensional Arrays, traversal
5. Strings: operations
6. Functions: call, return, passing parameters by (value, reference), passing arrays to function.
7. Recursion
8. Pointers: Pointers To Functions, Arrays, Strings, Pointers to Pointers, Array of Pointers
9. Structures: Nested Structures, Pointers to Structures, Arrays Structures and Unions.
10. Files: reading and writing, File pointers, file operations, random access, processor directives.

TOTAL: 60 PERIODS**COURSE OUTCOMES:**

Upon Completion Of The Course, the students will be able to

CO1: Demonstrate knowledge on C programming constructs. CO2:

Develop programs in C using basic constructs.

CO3: Develop programs in C using arrays.

CO4: Develop applications in C using strings, pointers, functions.

CO5: Develop applications in C using structures.

CO6: Develop applications in C using file processing.

TEXTBOOKS:

1. Reema Thareja, "Programming in C", Oxford University Press, Second Edition, 2016.
2. Kernighan, B. and Ritchie, D.M., "The C Programming language", Second Edition, Pearson Education, 2015.

REFERENCES:

1. Paul Deitel and Harvey Deitel, "C How to Program with an Introduction to C++", Eighth Edition, Pearson Education, 2018.
2. Yashwant Kanetkar, Let us C, 17th Edition, BPB Publications, 2020.
3. Byron S. Gottfried, "Schaum's Outline of Theory and Problems of Programming with C", McGraw-Hill Education, 1996.
4. Pradip Dey, Manas Ghosh, "Computer Fundamentals and Programming in C", Second Edition, Oxford University Press, 2013.
5. Anita Goel and Ajay Mittal, "Computer Fundamentals and Programming in C", 1st Edition, Pearson Education, 2013.

List of Experiments

Sl.No	Ex.No.	Date	Title Of The Experiments	Page No.	Staff sign
1.	1a		Programs using i/o statements		
2.	1b		Programs using operators and expressions		
3.	2a		Find odd or even using goto statement		
4.	2b		Finding leap year or not		
5.	2c		Creating a menu driven calculator		
6.	2d		Checking vowel or not using switch case		
7.	2e		Print 1 to n numbers except multiples of 5 using continue statement		
8.	3a		Finding armstrong number		
9.	3b		Sum of first n natural numbers using looping		
10	3c		Sum the cubes of series of n numbers		
11	4a		Finding sum of weights & sorting the elements based on weights		
12	4b		Finding the persons having above average height		
13	4c		Finding body mass index using array		
14	4d		Matrix addition and multiplication		
15	5		Reverse string without changing the position of special character		
16	6a		Swap the values using call by value		
17	6b		Sorting the values using pass by reference		
18	6c		Sum of array elements by passing to a function		
19	7a		Towers of Hanoi		
20	7b		Finding factorial of a number		
21	8a		Pointer to pointer		
22	8b		Array of Pointer		
23	9a		Employee Salary Slip		
24	9b		Students Internal MarkSheet		
25	9c		Accessing Union Members		
26	10a		Sequential Access File		
27	10b		Random Access File		
28	10c		Processor Directives		

AIM:

To Write a C program to use Input and Output Statements

ALGORITHM:

1. Start the program
2. Use Unformatted input & output statements to get char or string
 - getchar(),getche() ,getch() &gets()
3. Print the char or string using unformatted output statements
 - putchar(),putchar()&puts()
4. Use formatted Input statements read different types of data
 - scanf()
5. Print different types of data using formatted output statements
 - printf()
6. Stop the program

SYNTAX:**Unformatted I/O**

- charvariable=getchar();
- charvariable=getche();
- charvariable=getch();
- gets(chararray_variable);
- putchar(charvariable);
- puts(chararray_variable);

Formatted I/O

- scanf("formatspecifiers",addressofvariables);
- printf("Anytext");
- printf("Anytext,formatspcifiers",variables);
- printf("formatspcifiers",variables);

PROGRAM:

```
#include <stdio.h>
int main() {
    //Start the program
    printf("Starting the program...\n");
    int num;
    float fnum;
    char ch;
    char str[100];
```

```
// Use unformatted input statements

printf("Enter a character: ");
ch = getchar(); // Get a character from input
getchar(); // To consume the newline character

printf("Enter a string: ");
gets(str); // Get a string from input (note: gets is unsafe, use
           fgets in practice)

// Print the character and string using unformatted output
           statements
printf("You entered character: ");
putchar(ch); // Print the character
printf("\n");

printf("You entered string: ");
puts(str); // Print the string

// Use formatted input statements to read different types of data
printf("Enter an integer: ");
scanf("%d", &num); // Read an integer

printf("Enter a float: ");
scanf("%f", &fnum); // Read a float

// Print different types of data using formatted output
           statements
printf("You entered integer: %d\n", num); // Print the integer
printf("You entered float: %.2f\n", fnum); // Print the float with
           2 decimal places

// Stop the program
printf("Stopping the program...\n");
return 0;
}
```

OUTPUT:

Starting the program...
Enter a character: A
Enter a string: Hello, World!
You entered character: A
You entered string: Hello, World!
Enter an integer: 42
Enter a float: 3.14
You entered integer: 42
You entered float: 3.14
Stopping the program...

RESULT :

Thus the C program for I/O statements has been written & executed successfully.

AIM:

To Write C programs using Operators and Expressions

ALGORITHM:

1. Start the program
2. Use Arithmetic operators (+,-,*,/,%)
3. Use Increment & Decrement Operators(++,-)
4. Use comparison operators(>,<,>=,<=,==,!=)
5. Use Bitwise operators(~,^,|,&)
6. Use logical operators (||,&&,!)
7. Use Ternary Operator(?:)
8. Stop the program

SYNTAX:

Expression = Operand operator Operand

Operator : Symbol

Operand : variable or Constant

var1=Var2+Var3(or)Var1=Value1+Value2;

(or)

Var1=Var2+value

Logical Operators:

returns True (1) or false(0) values-Condition1 && Condition2

Ternary Operator:

If condition is true True part will be executed.

Otherwise, False part will be executed - Var1= (condition)? True part: False Part

PROGRAMS:

```
#include <stdio.h>
int main() {
    // Step 1: Start the program
    printf("Starting the program...\n");
    // Step 2: Use arithmetic operators
    int a = 10, b = 5;
    printf("Arithmetic Operations:\n");
    printf("Addition: %d + %d = %d\n", a, b, a + b);
    printf("Subtraction: %d - %d = %d\n", a, b, a - b);
    printf("Multiplication: %d * %d = %d\n", a, b, a * b);
    printf("Division: %d / %d = %d\n", a, b, a / b);
    printf("Modulus: %d %% %d = %d\n", a, b, a % b);
    // Step 3: Use increment and decrement operators
    printf("\nIncrement and Decrement Operations:\n");
    printf("Original a: %d\n", a);
    printf("a after increment: %d\n", ++a); // Pre-increment
    printf("a after decrement: %d\n", --a); // Pre-decrement
```

```

// Step 4: Use comparison operators
printf("\nComparison Operations:\n");
printf("Is a > b? %d\n", a > b);
printf("Is a < b? %d\n", a < b);
printf("Is a >= b? %d\n", a >= b);
printf("Is a <= b? %d\n", a <= b);
printf("Is a == b? %d\n", a == b);
printf("Is a != b? %d\n", a != b);
// Step 5: Use bitwise operators
printf("\nBitwise Operations:\n");
printf("Bitwise AND: %d & %d = %d\n", a, b, a & b);
printf("Bitwise OR: %d | %d = %d\n", a, b, a | b);
printf("Bitwise XOR: %d ^ %d = %d\n", a, b, a ^ b);
printf("Bitwise NOT: ~%d = %d\n", a, ~a);
// Step 6: Use logical operators
printf("\nLogical Operations:\n");
printf("Logical AND (true && false): %d\n", 1 && 0);
printf("Logical OR (true || false): %d\n", 1 || 0);
printf("Logical NOT (!true): %d\n", !1);
// Step 7: Use ternary operator
printf("\nTernary Operator:\n");
int max = (a > b) ? a : b; // Find maximum
printf("Maximum of %d and %d is %d\n", a, b, max);
// Step 8: Stop the program
printf("Stopping the program...\n");
return 0;
}

```

OUTPUT:

Starting the program...

Arithmetic Operations:

Addition: $10 + 5 = 15$

Subtraction: $10 - 5 = 5$

Multiplication: $10 * 5 = 50$

Division: $10 / 5 = 2$

Modulus: $10 \% 5 = 0$

Increment and Decrement Operations:

Original a: 10

A after increment: 11

A after decrement: 10

Comparison Operations:

Is $a > b$? 1

Is $a < b$? 0

Is $a >= b$? 1

Is $a <= b$? 1

Is $a == b$? 0

Is $a != b$? 1

Bitwise Operations:

Bitwise AND: $10 \& 5 = 0$

Bitwise OR: $10 | 5 = 15$

Bitwise XOR: $10 \wedge 5 = 15$

Bitwise NOT: $\sim 10 = -11$

Logical Operations:

Logical AND (true && false): 0

Logical OR (true || false): 1

Logical NOT (!True): 0

Ternary Operator:

Maximum of 10 and 5 is 10

Stopping the program...

RESULT:

Thus the C program for expressions has been written & executed successfully.

EX.NO:2a

FINDING ODD OR EVEN

AIM:

To Write a C program to find whether a given number is odd or even using goto statement.

ALGORITHM:

1. Start the program
2. Read an input Value
3. Divide it by 2.
4. Take the remainder,if the remainder is equal to zero ,then
5. Print the given Number is EVEN.
6. Otherwise, print the given Number is ODD.
7. Stop the Program.

SYNTAX:

```
if(condition)
    statement;
else
    statement;
```

PROGRAM:

```
#include <stdio.h>
#include<stdlib.h>

void main()
{
    Int num;

    printf("Enter a number\n");
    scanf("%d", &num);

    if(num%2==0)
        goto even;
    else
        goto odd;
```

even:

```
printf("%d is even\n",num);  
exit(0);
```

odd:

```
printf("%d is odd\n", num);  
}
```

OUTPUT 1:

Enter a number

74

74 is even

OUTPUT 2:

Enter a number

35

35 is odd

RESULT:

Thus the C program for finding even or odd is written & executed Successfully.

EX. NO: 2b**FINDING LEAP YEAR OR NOT****AIM:**

To write a C program to find whether the given year is a leap year or not.

ALGORITHM:

1. Start the program.
2. Read an input value (Year).
3. If the year is divisible by 4 and not divisible by 100, or it is divisible by 400, then:
4. Print the given year is a LEAP YEAR.
5. Otherwise, print the given year is NOT A LEAP YEAR.

PROGRAM:

```
#include <stdio.h>
int main() {
    int y;
    printf("Enter the year to check: ");
    scanf("%d", &y);

    if (((y % 4 == 0) && (y % 100 != 0)) || (y % 400 == 0))
        printf("It is a leap year.\n");
    else
        printf("It is not a leap year.\n");

    return 0;
}
```

OUTPUT:

Enter the year to check: 2024
It is a leap year.

RESULT:

Thus, the C program for finding whether the year is a leap year or not is written and executed successfully.

EX. NO: 2c**CREATING A MENU DRIVEN CALCULATOR****AIM:**

To write a C program for creating a menu-driven calculator using switch case.

ALGORITHM:

1. Start the program.
2. Read an option to select the operation:
 - 1. Add
 - 2. Subtract
 - 3. Multiply
 - 4. Divide
3. Enter the input values according to the operation.
4. Use switch case to perform the operation.
5. Print the result.
6. Stop the program.

PROGRAM:

```
#include <stdio.h>
#include <conio.h>

int main() {
    int option, a, b, c;
    clrscr(); // Clear the screen
    printf("Enter the option:");
    printf("\n1. ADD\t 2. SUB\t 3. MUL\t 4. DIV\n");
    scanf("%d", &option);
    switch(option) {
        case 1:
            printf("Enter two values: ");
            scanf("%d%d", &a, &b);
            c = a + b;
            break;
        case 2:
            printf("Enter two values: ");
            scanf("%d%d", &a, &b);
            c = a - b;
            break;
        case 3:
```

```

        printf("Enter two values: ");
        scanf("%d%d", &a, &b);
        c = a * b;
        break;
    case 4:
        printf("Enter two values: ");
        scanf("%d%d", &a, &b);
        c = a / b;
        break;
    default:
        printf("Choose the correct option\n");
        return 0; // Exit if option is incorrect
}

printf("Result is = %d\n", c);
getch();

return 0;
}

```

OUTPUT:

Enter the option:

1. ADD 2. SUB 3. MUL 4. DIV

1

Enter two values: 5 3

Result is = 8

RESULT:

Thus, the C program for creating a menu-driven calculator using switch case is written and executed successfully.

EX. NO: 2d CHECKING VOWEL OR NOT USING SWITCH CASE

AIM:

To write a C program for checking if a character is a vowel or not using switch case.

ALGORITHM:

1. Start the program.
2. Read a character input from the user.
3. Use a switch case to check if the character is a vowel (a, e, i, o, u).
4. Print whether the character is a vowel or not.
5. Stop the program.

PROGRAM:

```
#include <stdio.h>

int main() {
    char ch;
    printf("Enter a character: ");
    scanf(" %c", &ch); // Read a character input
    switch(ch) {
        case 'a':
        case 'e':
        case 'i':
        case 'o':
        case 'u':
        case 'A':
        case 'E':
        case 'I':
        case 'O':
        case 'U':
            printf("%c is a vowel.\n", ch);
            break;
        default:
            printf("%c is not a vowel.\n", ch);
    }
    return 0;
}
```

OUTPUT:

```
Enter a character: a
a is a vowel.
```

RESULT:

Thus, the C program for checking whether a character is a vowel or not using switch case is written and executed successfully.

EX.NO:2e

PRINT 1 TO N NUMBERS EXCEPT MULTIPLES OF 5

AIM:

To write a C program to print numbers from 1 to N, except multiples of 5, using the continue statement.

ALGORITHM:

1. Start the program.
2. Declare an integer variable 'N'.
3. Read the value of 'N' from the user.
4. Use a loop to iterate from 1 to 'N'.
 - If the current number is a multiple of 5, use the 'continue' statement to skip the current iteration.
5. Print the current number if it is not a multiple of 5.
6. End the program.

PROGRAM:

```
#include <stdio.h>

int main() {
    int N;
    printf("Enter a positive integer N: ");
    scanf("%d", &N); // Read the value of N

    // Loop from 1 to N
    for (int i = 1; i <= N; i++) {
        // Check if the number is a multiple of 5
        if (i % 5 == 0) {
            continue; // Skip the current iteration
        }
        // Print the number
        printf("%d ", i);
    }

    printf("\n"); // New line after printing all numbers
    return 0; // End the program
}
```

OUTPUT:

```
Enter a positive integer N: 15
1 2 3 4 6 7 8 9 11 12 13 14
```

RESULT:

Thus, the C program to print numbers from 1 to N, excluding multiples of 5 using the continue statement, is written and executed successfully.

EX.NO:3a**FINDING ARMSTRONG NUMBER****AIM:**

To Write a C program to find whether a given number is Armstrong number or not.

ALGORITHM:

1. Start the program.
2. Read an input value.
3. Split the digits using modulo division.
4. Cube the individual digits
5. Sum the cubes.
6. If the given number == sum of cubes, then
Print "ARMSTRONG NUMBER"
7. Otherwise,
Print "NOT AN ARMSTRONG NUMBER"
8. Stop the program.

PROGRAM:

```
#include <stdio.h>
#include <conio.h>

int main() {
    int N, A, digit, cube, sum = 0;
    printf("Enter a number: ");
    scanf("%d", &N);
    A = N; // Store the original number
    // Loop to calculate the sum of cubes of digits
    while (N > 0) {
        digit = N % 10; // Get the last digit
        cube = digit * digit * digit; // Calculate the cube of the digit
        sum = sum + cube; // Add the cube to sum
        N = N / 10; // Remove the last digit
    }
    // Check if the sum of cubes is equal to the original number
    if (sum == A)
        printf("%d is an Armstrong number.\n", A);
    else
        printf("%d is not an Armstrong number.\n", A);

    getch();
    return 0;
}
```

Input: $N = 153$, $A = 153$, $sum = 0$.

Iteration 1:

- $digit = 153 \% 10 = 3$
- $cube = 3*3*3 = 27$
- $sum = 0 + 27 = 27$
- $N = 153 / 10 = 15$

Iteration 2:

- $digit = 15 \% 10 = 5$
- $cube = 5*5*5 = 125$
- $sum = 27 + 125 = 152$
- $N = 15 / 10 = 1$

Iteration 3:

- $digit = 1 \% 10 = 1$
- $cube = 1*1*1 = 1$
- $sum = 152 + 1 = 153$
- $N = 1 / 10 = 0$

Loop ends ($N == 0$), compare sum (153) with A (153) $\rightarrow$ equal $\rightarrow$ prints 153 is an Armstrong number.

OUTPUT:

Enter a Number 153

153 is Armstrong Number

RESULT:

Thus the C program to find whether a given number is Armstrong number or not is written & executed successfully.

EX. NO: 3b SUM OF FIRST N NATURAL NUMBERS USING LOOPING

AIM:

To write a C program for calculating the sum of the first N natural numbers using different looping constructs: while loop, do...while loop, and for loop.

ALGORITHM:

Using While Loop

1. Start the program.
2. Declare an integer variable 'N' to hold the number of natural numbers.
3. Read the value of 'N' from the user.
4. Initialize a variable 'sum' to 0 and a counter variable 'counter' to 1.
5. While the 'counter' is less than or equal to 'N':
 - Add 'counter' to 'sum'.
 - Increment 'counter'.
6. Print the result of 'sum'.
7. Stop the program.

ALGORITHM:

Using Do...While Loop

1. Start the program.
2. Declare an integer variable 'N' to hold the number of natural numbers.
3. Read the value of 'N' from the user.
4. Initialize a variable 'sum' to 0 and a counter variable 'counter' to 1.
5. Use a do...while loop:
 - Add 'counter' to 'sum'.
 - Increment 'counter'.
 - Repeat until 'counter' is greater than 'N'.
6. Print the result of 'sum'.
7. Stop the program.

ALGORITHM:

Using For Loop

1. Start the program.
2. Declare an integer variable 'N' to hold the number of natural numbers.
3. Read the value of 'N' from the user.
4. Initialize a variable 'sum' to 0.
5. Use a for loop to iterate from 1 to 'N':
 - Add the current loop variable to 'sum'.
6. Print the result of 'sum'.
7. Stop the program.

PROGRAM 1:

```
#include <stdio.h>

int main() {
    int N, sum = 0, counter = 1;

    // Read the value of N
    printf("Enter a positive integer N: ");
    scanf("%d", &N);

    // Using while loop
    while (counter <= N) {
        sum += counter; // Add counter to sum
        counter++; // Increment counter
    }
    printf("Sum of first %d natural numbers (using while loop): %d\n", N, sum);
    return 0; // End the program
}
```

PROGRAM 2:

```
#include <stdio.h>

int main() {
    int N, sum = 0, counter = 1;

    // Read the value of N
    printf("Enter a positive integer N: ");
    scanf("%d", &N);

    // Using do...while loop
    do {
        sum += counter; // Add counter to sum
        counter++; // Increment counter
    } while (counter <= N);

    printf("Sum of first %d natural numbers (using do...while loop): %d\n", N, sum);

    return 0; // End the program
}
```

PROGRAM 3:

```
#include <stdio.h>

int main() {
    int N, sum = 0;

    // Read the value of N
    printf("Enter a positive integer N: ");
    scanf("%d", &N);

    // Using for loop
    for (int counter = 1; counter <= N; counter++) {
        sum += counter; // Add counter to sum
    }

    printf("Sum of first %d natural numbers (using for loop): %d\n", N, sum);

    return 0; // End the program
}
```

OUTPUT:

Enter a positive integer N: 5
Sum of first 5 natural numbers (using while loop): 15

Enter a positive integer N: 5
Sum of first 5 natural numbers (using do...while loop): 15

Enter a positive integer N: 5
Sum of first 5 natural numbers (using for loop): 15

RESULT:

Thus, the C program for calculating the sum of the first N natural numbers using while, do...while, and for loops is written and executed successfully.

EX. NO: 3c**SUM THE CUBES OF SERIES OF N NUMBERS****AIM:**

To write a C program to sum the cubes of a series of N numbers.

ALGORITHM:

1. Start the program.
2. Read the input from the user and store it in the variable `n`.
3. Assign `sum = 0`.
4. Iterate the loop for `n` times, cubing each number and taking the sum of them.
5. Print the sum.
6. Stop the program.

PROGRAM:

```
#include <stdio.h>

int main() {
    int n;
    int sum = 0;

    printf("Enter the number: ");
    scanf("%d", &n);

    for (int i = 1; i <= n; i++)
        sum += i * i * i; // Cube of i

    printf("Sum = %d\n", sum);
    return 0; // End the program
}
```

1. `#include <stdio.h>`
 - Brings in the Standard I/O library so you can use `printf` and `scanf`.
2. `int main() {`
 - Entry point of the program. Execution starts here.
3. `int n;`
 - Declares an integer variable `n` that will hold the user's input (the upper limit).
4. `int sum = 0;`
 - Declares and initializes `sum` to 0. `sum` will accumulate the sum of cubes from 1 to `n`.
5. `printf("Enter the number: ");`
 - Prints a prompt asking the user to enter a number.
6. `scanf("%d", &n);`
 - Reads an integer from standard input and stores it in `n`. (Good practice: check the return value of `scanf` to ensure successful input.)
7. `for (int i = 1; i <= n; i++)`
 - A for loop that sets `i` to 1 initially, runs while `i` is $\leq n$, and increments `i` by 1 each iteration. Each iteration processes the next integer from 1 to `n`.
8. `sum += i * i * i; // Cube of i`
 - Inside the loop: compute `i*i*i` (the cube of `i`) and add it to `sum`. The `+=` is shorthand for `sum = sum + (i*i*i);`
 - Note: Because there are no braces `{ }`, this single statement is the only one inside the loop.

9. `printf("Sum = %d\n", sum);`
 - After the loop finishes, print the final value of sum.
10. `return 0;`
 - Exit the program and return 0 to the operating system (convention for successful completion).

Concrete example

If the user inputs $n = 3$:

- Iteration $i=1 \rightarrow \text{add } 1*1*1 = 1 \rightarrow \text{sum} = 1$
 - Iteration $i=2 \rightarrow \text{add } 2*2*2 = 8 \rightarrow \text{sum} = 9$
 - Iteration $i=3 \rightarrow \text{add } 3*3*3 = 27 \rightarrow \text{sum} = 36$
- Program prints: Sum = 36

This is exact and much faster for large n

OUTPUT:

Enter the number: 3
Sum = 36

RESULT:

Thus, the C program for summing the cubes of a series of N numbers has been executed successfully and the output was verified.

EX. NO: 4a**FINDING SUM OF WEIGHTS & SORTING THE ELEMENTS BASED ON WEIGHTS****AIM:**

Write a C program to find the sum of weights for a given set of numbers like <10, 36, 54, 89, 12, 27>. Find the sum of weights based on the following conditions:

- 1) 5 if it is a perfect cube.
- 2) 4 if it is a multiple of 4 and divisible by 6.
- 3) 3 if it is a prime number.

ALGORITHM:

1. Declare and initialize an integer array `a[]`.
2. Declare an array called `w[]` to store the weight values of the respective elements.
3. Assign the weight values to each element of that array by checking the conditions:
 - If `a[i]` is a perfect cube, assign weight 5 to `w[i]`.
 - If `a[i]` is a multiple of 4 & divisible by 6, assign 4 to `w[i]`.
 - If `a[i]` is a prime number, assign 3 to `w[i]`.
4. Add all weight values.
5. Print the sum of weights.
6. Sort the array elements in increasing order based on their weight values.

PROGRAM:

```
#include <stdio.h>
#include <math.h>

int is_prime(int num) {
    if (num <= 1) return 0;
    for (int j = 2; j <= sqrt(num); j++) {
        if (num % j == 0) return 0;
    }
    return 1;
}

int main() {
    int a[10] = {10, 36, 54, 89, 12, 27};
    int w[10] = {0};
    int N = 6, sum = 0;

    // Calculate weights
    for (int i = 0; i < N; i++) {
        int A = a[i];
        double cubroot = round(pow(A, 1.0/3));
        int cube = (int)(cubroot * cubroot * cubroot);

        if (A == cube) {
            w[i] += 5; // Perfect cube
        }
        if (A % 4 == 0 && A % 6 == 0) {
            w[i] += 4; // Multiple of 4 and divisible by 6
        }
    }
}
```

```

    if (is_prime(A)) {
        w[i] += 3; // Prime number
    }
}

// Print elements and their weights
printf("\nELEMENT\t\tWEIGHT\n");
for (int i = 0; i < N; i++) {
    printf("%d\t\t%d\n", a[i], w[i]);
    sum += w[i];
}

printf("The sum of weights is: %d\n", sum);

// Sorting the elements based on weight values
for (int i = 0; i < N - 1; i++) {
    for (int j = 0; j < N - i - 1; j++) {
        if (w[j] > w[j + 1]) {
            // Swap weights
            int temp_w = w[j];
            w[j] = w[j + 1];
            w[j + 1] = temp_w;
            // Swap corresponding elements
            int temp_a = a[j];
            a[j] = a[j + 1];
            a[j + 1] = temp_a;
        }
    }
}

// Print sorted elements with their weights
printf("Sorting of the elements based on weight values:\n");
for (int i = 0; i < N; i++) {
    printf("<%d, %d>\t", a[i], w[i]);
}

return 0; // End the program
}

```

OUTPUT:

ELEMENT	WEIGHT
10	0
36	4
54	0
89	3
12	4
27	5

The sum of weights is: 16

Sorting of the elements based on weight values:

<10, 0> <54, 0> <89, 3> <36, 4> <12, 4> <27, 5>

RESULT:

Thus, the C program for finding the sum of weights and sorting the elements based on weights has been executed successfully, and the output was verified.

Ex.No:4b FINDING THE PERSONS HAVING ABOVE AVERAGE HEIGHT

AIM:

Write a C program to populate an array with height of persons and find how many persons are above the average height.

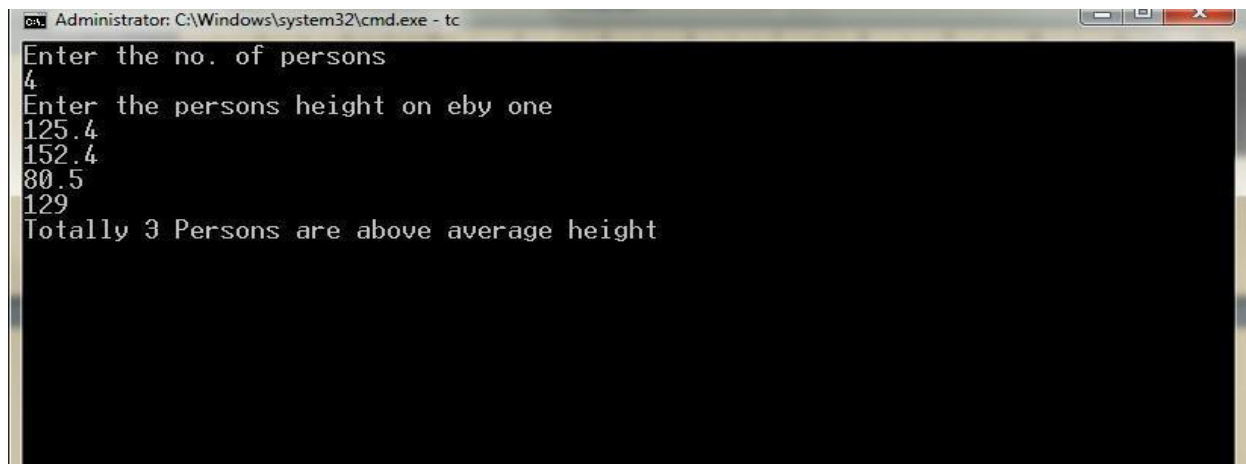
ALGORITHM:

1. Declare an array to store the height values of persons
2. Initialize the count value to 0.
3. Read the number of persons
4. Read the individual person's height
5. Find the sum of heights
6. Find the average of heights
7. Check whether a person's height is greater than the average height or not.
 1. If Yes, then increase the count value
8. Print the count value.

PROGRAM:

```
#include<stdio.h>
#include<conio.h>
void main()
{
float
height[20],avg;intN,i,s
um=0,count=0;
clrscr();
printf("Entertheno.ofpersons");
scanf("%d",&N);
printf("Enterthepersonsheightonebyone\n");
for(i=0;i<N;i++)
{
scanf("%f",&height[i]);
sum=sum+height[i];
}
avg=sum/N;
for(i=0;i<N;i++)
{
if(height[i]>avg)
{ count=count+1;}
}
printf("Totally%dPersonsarehavingaboveaverageheight",count);
getch();
}
```

OUTPUT:



```
Administrator: C:\Windows\system32\cmd.exe - tc
Enter the no. of persons
4
Enter the persons height on eby one
125.4
152.4
80.5
129
Totally 3 Persons are above average height
```

RESULT:

Thus the program is written & executed Successfully.

EX.NO:4c

FINDING BODY MASS INDEX USING ARRAY

AIM:

To write a C program to perform the following:

- Populate a two-dimensional array with the height and weight of persons.
- Compute the Body Mass Index of the individuals.

ALGORITHM:

1. Start the program.
2. Read the number of persons.
3. Enter the height and weight values.
4. Read height in centimeters and weight in kilograms of a person.
5. Height in meters = height in cm / 100;
6. Compute the Body Mass Index (BMI) using the following formula:
 $BMI = (\text{weight in Kg}) / (\text{Height in Meters})^2$
7. Store the BMI values in a resultant array.
8. Print the results.
9. Stop the program

PROGRAM:

```
#include<stdio.h>
#include<conio.h>
void main()
{
floatheight[20],weight[20],BMI[20],HIM[20];
int i,j,N;
clrscr();
printf("\nEntertheNo.oftheelements\n");
scanf("%d",&N);
printf("\nEntertheHeight&Weightvalues\n");
for(i=0;i<N;i++)
{
scanf("%f%f",&height[i],&weight[i]);
HIM[i]=height[i]/100;
}
printf("\nPerson\tHeight\tWeight\tBMS\n");
for(i=0;i<N;i++)
{
BMI[i]=weight[i]/(HIM[i]*HIM[i]);
printf("\n%d\t%.2f\t%.2f\t%.2f\n",(i+1),HIM[i],weight[i],BMI[i]);
}
}
getch();
}
```

OUTPUT:

Enter No. of the elements 3

Enter the Height & Weight values

152 58

122 28

135 40

Person	Height	Weight	BMS
1	1.52	58.00	25.10
2	1.22	28.00	18.81
3	1.35	40.00	21.95

RESULT:

Thus the program is written & executed Successfully.

EX.NO:4d

MATRIX ADDITION AND MULTIPLICATION

AIM:

To write a C program to calculate matrix addition and multiplication.

ALGORITHM 1:

Matrix Addition

1. Start the program.
2. Enter the number of rows r and columns c .
3. Enter the elements of the two matrices (of order $r \times c$).
4. Add the corresponding elements of the two matrices and save it in another matrix (two-dimensional array).
5. Print the resultant matrix.
6. Stop the program.

ALGORITHM 2:

Matrix Multiplication

1. Start the program.
2. Read the size (rows and columns) of the two matrices.
3. To multiply two matrices, the number of columns of the first matrix should be equal to the number of rows of the second matrix.
4. Multiply the two matrices by using the following three functions:
 - **getMatrixElements()** - to take matrix elements input from the user.
 - **multiplyMatrices()** - to multiply two matrices.
 - **display()** - to display the resultant matrix after multiplication.
5. Stop the program.

PROGRAM:1

```
#include <stdio.h>
```

```
int main() {
```

```
    int r, c, a[10][10], b[10][10], sum[10][10], i, j;
```

```
    printf("Enter the number of rows (between 1 and 10): ");
```

```
    scanf("%d", &r);
```

```
    printf("Enter the number of columns (between 1 and 10): ");
```

```
    scanf("%d", &c);
```

```

printf("\nEnter elements of 1st matrix:\n");
for (i = 0; i < r; ++i) {
    for (j = 0; j < c; ++j) {
        printf("Enter element a%d%d: ", i + 1, j + 1);
        scanf("%d", &a[i][j]);
    }
}

printf("Enter elements of 2nd matrix:\n");
for (i = 0; i < r; ++i) {
    for (j = 0; j < c; ++j) {
        printf("Enter element b%d%d: ", i + 1, j + 1);
        scanf("%d", &b[i][j]);
    }
}

// Adding two matrices
for (i = 0; i < r; ++i) {
    for (j = 0; j < c; ++j) {
        sum[i][j] = a[i][j] + b[i][j];
    }
}

// Printing the result
printf("\nSum of two matrices:\n");
for (i = 0; i < r; ++i) {
    for (j = 0; j < c; ++j) {
        printf("%d ", sum[i][j]);
    }
    printf("\n");
}

return 0;
}

```

PROGRAM:2

```
#include <stdio.h>

// Function to get matrix elements entered by the user
void getMatrixElements(int matrix[][10], int row, int column) {
    printf("\nEnter elements: \n");
    for (int i = 0; i < row; ++i) {
        for (int j = 0; j < column; ++j) {
            printf("Enter a %d%d: ", i + 1, j + 1);
            scanf("%d", &matrix[i][j]);
        }
    }
}

// Function to multiply two matrices
void multiplyMatrices(int first[][10], int second[][10], int result[][10], int r1, int c1, int r2, int c2) {
    // Initializing elements of result matrix to 0
    for (int i = 0; i < r1; ++i) {
        for (int j = 0; j < c2; ++j) {
            result[i][j] = 0;
        }
    }

    // Multiplying first and second matrices and storing it in result
    for (int i = 0; i < r1; ++i) {
        for (int j = 0; j < c2; ++j) {
            for (int k = 0; k < c1; ++k) {
                result[i][j] += first[i][k] * second[k][j];
            }
        }
    }
}

// Function to display the matrix
void display(int result[][10], int row, int column) {
    printf("\nOutput Matrix:\n");
    for (int i = 0; i < row; ++i) {
        for (int j = 0; j < column; ++j) {
            printf("%d ", result[i][j]);
            if (j == column - 1) printf("\n");
        }
    }
}

int main() {
    int first[10][10], second[10][10], result[10][10], r1, c1, r2, c2;
    printf("Enter rows and columns for the first matrix: ");
    scanf("%d %d", &r1, &c1);
    printf("Enter rows and columns for the second matrix: ");
    scanf("%d %d", &r2, &c2);
```

```

// Taking input until 1st matrix columns is not equal to 2nd matrix rows
while (c1 != r2) {
    printf("Error! Enter rows and columns again.\n");
    printf("Enter rows and columns for the first matrix: ");
    scanf("%d %d", &r1, &c1);
    printf("Enter rows and columns for the second matrix: ");
    scanf("%d %d", &r2, &c2);
}

// Get elements of the first matrix
getMatrixElements(first, r1, c1);

// Get elements of the second matrix
getMatrixElements(second, r2, c2);

// Multiply two matrices
multiplyMatrices(first, second, result, r1, c1, r2, c2);

// Display the result
display(result, r1, c2);

return 0;
}

```

OUTPUT :1

Enter the number of rows (between 1 and 10): 2
Enter the number of columns (between 1 and 10): 3

Enter elements of 1st matrix:

Enter element a11: 2
Enter element a12: 3
Enter element a13: 4
Enter element a21: 5
Enter element a22: 2
Enter element a23: 3

Enter elements of 2nd matrix:

Enter element b11: -4
Enter element b12: 5
Enter element b13: 3
Enter element b21: 5
Enter element b22: 6
Enter element b23: 3

Sum of two matrices:

-2 8 7
10 8 6

OUTPUT :2

Enter rows and columns for the first matrix: 2 3
Enter rows and columns for the second matrix: 3 2

Enter elements:

Enter a11: 2

Enter a12: -3

Enter a13: 4

Enter a21: 5

Enter a22: 3

Enter a23: 5

Enter elements:

Enter a11: 3

Enter a12: 3

Enter a21: 5

Enter a22: 0

Enter a31: -3

Enter a32: 4

Output Matrix:

-2 12

19 7

RESULT:

Thus, the C program to calculate matrix addition and multiplication has been executed successfully and the output was verified.

EX.NO: 5**REVERSE STRING WITHOUT CHANGING THE POSITION OF SPECIAL CHARACTER****AIM:**

To write a C program for reversing a string without changing the position of special characters.
(Example input: a@gh%;j and output: j@hg%;a)

ALGORITHM:

1. Start the program.
2. Read a string.
3. Call a function to reverse a string.
4. Reverse function:
 - Find string length.
 - Set forward & reverse pointers (f = 0, r = strlen - 1).
 - Check the following:
 - If both str[f] and str[r] are alphanumeric, then swap these two chars in that string and do f++, r--.
 - If str[f] is a special char, then f++.
 - If str[r] is a special char, then r--.
 - If both are special chars, then f++ & r--.
5. Print the reversed string.
6. Stop the program.

PROGRAM:

```
#include <stdio.h>
#include <conio.h>
void reverse(char*);

void main() {
    char str[50];
    clrscr();
    printf("Enter a string\n");
    scanf("%s", str);
    printf("Input string: %s\n", str);
    reverse(str);
    printf("Output string: %s\n", str);
    getch();
}
```

```

}
void reverse(char* str) {
    int r = strlen(str) - 1, f = 0;
    char t;
    while (f < r) {
        if (isalnum(str[f]) != 0 && isalnum(str[r]) != 0) {
            t = str[r];
            str[r] = str[f];
            str[f] = t;
            f++;
            r--;
        } else if (isalnum(str[f]) != 0 && isalnum(str[r]) == 0) {
            r--;
        } else if (isalnum(str[f]) == 0 && isalnum(str[r]) != 0) {
            f++;
        } else {
            f++;
            r--;
        }
    }
}
}

```

OUTPUT:

Enter a string: a@gh%;j

Input string: a@gh%;j

Output string: j@hg%;a

RESULT:

Thus, the C program to reverse a string without changing the position of special characters has been executed successfully and the output was verified.

EX.NO: 6a

SWAP THE VALUES USING CALL BY VALUE

AIM:

To write a C program for swapping two numbers using call by value.

ALGORITHM:

1. Start the program.
2. Read values from the user.
3. Swap the values of two variables using a third variable.
4. Print two values after swapping.
5. Stop the program.

PROGRAM:

```
#include <stdio.h>

void swap(int, int);

int main() {
    int x, y;
    printf("Enter the value of x and y\n");
    scanf("%d%d", &x, &y);
    printf("Before Swapping\nx = %d\nny = %d\n", x, y);
    swap(x, y);
    printf("After Swapping\nx = %d\nny = %d\n", x, y);
    return 0;
}

void swap(int a, int b) {
    int temp;
    temp = b;
    b = a;
    a = temp;
    printf("Values of a and b are %d %d\n", a, b);
}
```

OUTPUT:

Enter the value of x and y

Before Swapping

x = 10

y = 5

Values of a and b are 5 10

After Swapping

x = 10

y = 5

RESULT:

Thus, the C program for swapping two numbers using call by value has been executed successfully and the output was verified.

EX.NO: 6b SORT THE LIST OF NUMBERS USING PASS BY REFERENCE

AIM:

Write a C program to sort numbers using pass by reference.

ALGORITHM:

1. Start the program.
2. Read the size of the array.
3. Read the elements one by one.
4. Call the function sort().
5. Pass the starting address of the array to the sort function as input.
6. Store the address in the formal argument of that function (pointer variable).
 - Access the array values by increasing the addresses of the pointer variable.
 - Sort the values using the pointer variable.
7. Print the result.
8. Stop the program.

PROGRAM:

```
#include <stdio.h>

void sort(int*, int);

void main() {
    int a[20], N, i;
    printf("Enter the no. of elements\n");
    scanf("%d", &N);
    printf("Enter the Elements one by one\n");
    for (i = 0; i < N; i++)
        scanf("%d", &a[i]);
    sort(a, N);
    printf("\nSorted Order\n");
    for (i = 0; i < N; i++)
        printf("%d\t", a[i]);
    getch();
}
```

```
void sort(int* x, int n) {  
    int t, i, j;  
    for (i = 0; i < n - 1; i++) {  
        for (j = 0; j < n - 1; j++) {  
            if (*(x + j) > *(x + (j + 1))) {  
                t = *(x + j);  
                *(x + j) = *(x + (j + 1));  
                *(x + (j + 1)) = t;  
            }  
        }  
    }  
}
```

OUTPUT:

Enter the no. of elements: 3

Enter the Elements one by one:

100

3

45

Sorted Order:

3 45 100

RESULT:

Thus, the C program for sorting the values using pass by reference has been executed successfully and the output was verified.

EX.NO: 6c SUM OF ARRAY ELEMENTS BY PASSING TO A FUNCTION

AIM:

Write a C program Sum of Array Elements by Passing to a Function.

ALGORITHM:

1. Start the program.
2. Read the size of the array (n)
3. Declare an array of size n
4. Input all the elements of the array
5. Call a function sumArray(arr, n) by passing the array and its size as arguments
6. Inside the function:
7. Initialize sum = 0
8. Repeat for $i = 0$ to $n-1$
9. $sum = sum + arr[i]$
10. Return sum to the calling function
11. Display the result (sum of array elements)
12. Stop

PROGRAM:

```
#include <stdio.h>

int sumArray(int arr[], int n)
{
    int sum = 0, i;
    for(i = 0; i < n; i++)
    {
        sum += arr[i];
    }
    return sum;
}

int main()
{
    int n, i;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d elements: ", n);
    for(i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }
    int result = sumArray(arr, n);
    printf("Sum of array elements = %d\n", result);
    return 0;
}
```

OUTPUT:

Enter the number of elements: 5
Enter 5 elements: 10 20 30 40 50
Sum of array elements = 150

RESULT:

Thus, the C program for Sum of Array Elements by Passing to a Function has been executed successfully and the output was verified.

EX.NO: 7a

TOWERS OF HANOI

AIM:

To write a C program to solve the Towers of Hanoi using recursion.

ALGORITHM:

1. Start the program.
2. Create a function called `towers()`.
3. Read the number of disks.
4. Call the function `towers()` to move the disks from source to destination.
5. Stop the program.

PROGRAM:

```
#include <stdio.h>

void towers(int num, char frompeg, char topeg, char auxpeg) {
    if (num == 1) {
        printf("\nMove disk 1 from peg %c to peg %c", frompeg, topeg);
        return;
    }
    towers(num - 1, frompeg, auxpeg, topeg);
    printf("\nMove disk %d from peg %c to peg %c", num, frompeg, topeg);
    towers(num - 1, auxpeg, topeg, frompeg);
}

int main() {
    int num;
    printf("Enter the number of disks: ");
    scanf("%d", &num);
    printf("The sequence of moves involved in the Tower of Hanoi are:");
    towers(num, 'A', 'C', 'B');
    return 0;
}
```

OUTPUT:

```
Enter the number of disks: 3
The sequence of moves involved in the Tower of Hanoi are:
Move disk 1 from peg A to peg C
Move disk 2 from peg A to peg B
Move disk 1 from peg C to peg B
Move disk 3 from peg A to peg C
Move disk 1 from peg B to peg A
Move disk 2 from peg B to peg C
Move disk 1 from peg A to peg C
```

RESULT:

The program has been executed successfully, and the sequence of moves has been verified.

EX.NO: 7b

FINDING FACTORIAL OF A NUMBER

AIM:

To write a C program for finding the factorial of a number using recursion.

ALGORITHM:

1. Start the program.
2. Ask the user to enter an integer to find the factorial.
3. Read the integer and assign it to a variable.
4. Multiply each digit recursively from the integer down to 1.
5. The final value is the factorial.
6. Stop the program.

PROGRAM:

```
#include <stdio.h>

long factorial(int n) {
    if (n == 0) return 1;
    else return n * factorial(n - 1);
}

int main() {
    int number;
    long fact;
    printf("Enter a number: ");
    scanf("%d", &number);
    fact = factorial(number);
    printf("Factorial of %d is %ld\n", number, fact);
    return 0;
}
```

OUTPUT:

```
Enter a number: 6
Factorial of 6 is 720
```

RESULT:

The C program for finding the factorial of a number using recursion has been executed successfully, and the output was verified.

EX.NO: 8a

POINTER TO POINTER

AIM:

To write a C program to print the value and address of a variable using pointer to pointer.

ALGORITHM:

1. Start the program.
2. Declare an integer variable and a pointer to it.
3. Declare a pointer to the pointer.
4. Print the addresses and values.

PROGRAM:

```
#include <stdio.h>

int main() {
    int a = 10;
    int *p;
    int **pp;

    p = &a;    // Pointer p points to the address of a
    pp = &p;   // Pointer pp is a double pointer pointing to the address of p

    printf("Address of a: %p\n", (void*)&a); // Address of a
    printf("Address of p: %p\n", (void*)p); // Address stored in p
    printf("Value stored at p: %d\n", *p);  // Value at the address contained by p
    printf("Value stored at pp: %d\n", **pp); // Value at the address contained by pp
    return 0;
}
```

OUTPUT:

```
Address of a: 0x7ffedc8a3c8
Address of p: 0x7ffedc8a3c4
Value stored at p: 10
Value stored at pp: 10
```

RESULT:

The C program to print the value and address of a variable using pointer to pointer has been executed successfully, and the output was verified.

EX.NO: 8b

ARRAY OF POINTER

AIM:

To write a C program to read and display 5 students' names using an array of pointers.

ALGORITHM:

1. Start the program.
2. Declare an array of pointers to store names.
3. Read the names.
4. Display the names.

PROGRAM:

```
#include <stdio.h>

int main() {
    const int MAX = 5;
    char *names[MAX] = { "Zara Ali", "Hina Ali", "Nuha Ali", "Sara Ali", "Sameer Ali" };
    int i;

    for (i = 0; i < MAX; i++) {
        printf("Value of names[%d] = %s\n", i, names[i]);
    }

    return 0;
}
```

OUTPUT:

```
Value of names[0] = Zara Ali
Value of names[1] = Hina Ali
Value of names[2] = Nuha Ali
Value of names[3] = Sara Ali
Value of names[4] = Sameer Ali
```

RESULT:

The C program to read and display 5 student names using an array of pointers has been executed successfully, and the output was verified.

EX.NO: 9a

EMPLOYEE SALARY SLIP

AIM:

To write a C program to generate an employee salary slip using structure and pointer.

ALGORITHM:

1. Start the program.
2. Create a structure called 'Employee' with fields for employee ID, name, department, designation, and salary.
3. Create a pointer to a structure.
4. Call a function to generate the salary slip for a particular employee.
5. Stop the program.

PROGRAM:

```
#include <stdio.h>

struct Employee {
    char name[25];
    int eid;
    char dept[20];
    char des[20];
    int salary;
};

void salarySlip(struct Employee *e, int n) {
    int id, i;
    printf("\nEnter the employee ID to generate the salary slip: ");
    scanf("%d", &id);

    for (i = 0; i < n; i++) {
        if ((e + i)->eid == id) {
            printf("\nNAME\t\tDEPARTMENT\t\tDESIGNATION\t\tSALARY");
            printf("\n%s\t\t%s\t\t%s\t\t%d", (e + i)->name, (e + i)->dept, (e + i)->des, (e + i)->salary);
            printf("\n");
            return; // Exit after finding the employee
        }
    }
    printf("Employee ID not found.\n");
}

int main() {
    struct Employee emp[20], *emp1;
    int m, i;

    printf("Enter the number of employee details: ");
    scanf("%d", &m);

    printf("\nEnter employee ID, name, department, designation & salary:\n");
    for (i = 0; i < m; i++) {
        scanf("%d %s %s %s %d", &emp[i].eid, emp[i].name, emp[i].dept, emp[i].des, &emp[i].salary);
    }
}
```

```
    emp1 = emp;
    salarySlip(emp1, m);

    return 0;
}
```

OUTPUT:

Enter the number of employee details: 2

Enter employee ID, name, department, designation & salary:

101 A CSE AP 20000

202 B ECE AP 25000

Enter the employee ID to generate the salary slip: 101

NAME	DEPARTMENT	DESIGNATION	SALARY
ARUN	CSE	AP	20000

RESULT:

The C program to generate employee salary slips using structure and pointer has been executed successfully, and the output was verified.

AIM:

To write a C program to compute internal marks of students for five different subjects using nested structure.

ALGORITHM:

1. Start the program.
2. Create a structure called `Student` with fields for name, test marks for subjects, and internal marks.
3. Create a function called `calculateInternal()`.
4. Call the function to compute and print the marks.
5. Stop the program.

PROGRAM:

```
#include <stdio.h>

struct Student {
    char name[20];
    int marks[3][5]; // 3 tests for 5 subjects
    int internalMarks[5];
};

void calculateInternal(struct Student s) {
    int j, k, total;

    for (j = 0; j < 5; j++) {
        total = 0;
        for (k = 0; k < 3; k++) {
            total +=

s.marks[k][j];
        }
        s.internalMarks[j] = (total / 3) * 0.2; // Assuming max marks 20
    }

    for (j = 0; j < 5; j++) {
        printf("Subject %d Internal Mark (max marks 20) = %d\n", j + 1, s.internalMarks[j]);
    }
}

int main() {
    struct Student s;
    int j, k;

    for (j = 0; j < 3; j++) {
        printf("Enter the internal test %d marks for five subjects:\t", j + 1);
        for (k = 0; k < 5; k++) {
            scanf("%d", &s.marks[j][k]);
        }
    }
    calculateInternal(s);

    return 0;
}
```

OUTPUT:

Enter the internal test 1 marks for five subjects: 10 10 10 10 10
Enter the internal test 2 marks for five subjects: 70 80 90 100 80
Enter the internal test 3 marks for five subjects: 90 90 90 90 90

Subject 1 Internal Mark (max marks 20) = 12
Subject 2 Internal Mark (max marks 20) = 12
Subject 3 Internal Mark (max marks 20) = 12
Subject 4 Internal Mark (max marks 20) = 14
Subject 5 Internal Mark (max marks 20) = 12

RESULT:

The C program to compute internal marks of students for five different subjects using nested structure has been executed successfully, and the output was verified.

AIM:

To write a C program for accessing student details using a union.

ALGORITHM:

1. Start the program.
2. Define a union for student details, including fields for the student's ID, name, and grade.
3. Read the student details from the user.
4. Print the student details.
5. Stop the program.

PROGRAM:

```
#include <stdio.h>
#include <string.h>

// Define a union for student details
union Student {
    int id;
    char name[50];
    float grade;
};

int main() {
    union Student student;

    // Accessing student ID
    printf("Enter Student ID: ");
    scanf("%d", &student.id);
    printf("Student ID: %d\n", student.id);

    // Accessing student name
    printf("Enter Student Name: ");
    // Clear the input buffer
    getchar(); // To consume the newline character left by scanf
    fgets(student.name, sizeof(student.name), stdin);
    // Remove the newline character from the name
    student.name[strcspn(student.name, "\n")] = '\0';
    printf("Student Name: %s\n", student.name);

    // Accessing student grade
    printf("Enter Student Grade: ");
    scanf("%f", &student.grade);
    printf("Student Grade: %.2f\n", student.grade);

    // Note: Only the last accessed member (grade) will have a valid value
    printf("Accessing ID after Grade: %d (may be garbage)\n", student.id);
    printf("Accessing Name after Grade: %s (may be garbage)\n", student.name);

    return 0;
}
```

OUTPUT:

Enter Student ID: 123
Student ID: 123
Enter Student Name: John Doe
Student Name: John Doe
Enter Student Grade: 85.5
Student Grade: 85.50
Accessing ID after Grade: 123 (may be garbage)
Accessing Name after Grade: John Doe (may be garbage)

RESULT:

This C program successfully demonstrates how to use a union to access student details.

EX.NO: 10a**SEQUENTIAL ACCESS FILE****AIM:**

To write a C program to count the number of account holders whose balance is less than the minimum balance using a sequential access file.

ALGORITHM:

1. Start the program.
2. Read choice to insert records or count minimum balance accounts.
3. If choice is 1, then:
 - Open a data file in write mode.
 - Read the number of records.
 - Write the records into the file using `fprintf()`.
 - Close the file.
4. If choice is 2, then:
 - Open the file in read mode.
 - Read the records one by one using `fscanf()` until reaching the end of the file.
 - Check the account balance against the minimum balance.
 - If the account balance is less than the minimum balance, display the account details.
 - Close the file.
5. Stop the program.

PROGRAM:

```
#include <stdio.h>
void insert();
void count();
int main(void) {
    int choice = 0;
    while (choice != 3) {
        printf("\n1. Insert records\n");
        printf("2. Count min balance holders\n");
        printf("3. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1: insert(); break;
            case 2: count(); break;
        }
    }
}

void insert() {
    unsigned int account, i;
    char name[30];
    double balance;
    FILE* cfPtr;
    if ((cfPtr = fopen("clients.dat", "w")) == NULL) {
        puts("File could not be opened");
    } else {
        int records, i = 0;
        printf("Enter the number of records: ");
        scanf("%d", &records);
        while (i < records) {
            printf("Enter the account, name, and balance: ");
```

```

        scanf("%d %29s %lf", &account, name, &balance);
        fprintf(cfPtr, "%d %s %.2f\n", account, name, balance);
        i++;
    }
    fclose(cfPtr);
}
}
void count() {
    unsigned int account;
    char name[30];
    double balance;
    float minBal = 5000.00;
    int count = 0;
    FILE* cfPtr;
    if ((cfPtr = fopen("clients.dat", "r")) == NULL) {
        printf("File could not be opened");
    } else {
        printf("%-10s%-13s%s\n", "Account", "Name", "Balance");
        fscanf(cfPtr, "%d %29s %lf", &account, name, &balance);
        while (!feof(cfPtr)) {
            if (balance < minBal) {
                printf("%-10d%-13s%7.2f\n", account, name, balance);
                count++;
            }
            fscanf(cfPtr, "%d %29s %lf", &account, name, &balance);
        }
        fclose(cfPtr);
        printf("The number of account holders whose balance is less than the minimum balance:
%d\n", count);
    }
}
}

```

OUTPUT:

```

1. Insert records
2. Count min balance holders
3. Exit
Enter choice: 1
Enter the number of records: 2
Enter the account, name, and balance: 1001 A 10000
Enter the account, name, and balance: 1002 B 300

```

```

1. Insert records
2. Count min balance holders
3. Exit
Enter choice: 2
Account      Name      Balance
1002      B      300.00
The number of account holders whose balance is less than the minimum balance: 1

```

RESULT:

The C program to count the number of account holders whose balance is less than the minimum balance using a sequential access file has been executed successfully, and the output was verified.

EX.NO: 10b**RANDOM ACCESS FILE****AIM:**

To write a C program to update telephone details of an individual or a company in a telephone directory using a random access file.

ALGORITHM:

1. Start the program.
2. Store the telephone details into a file.
3. Read the data and display it.
4. Enter the telephone number to be modified and the new number.
5. Use `fseek()` function to randomly access the record.
6. Copy the contents from the source file to the destination file.
7. Store the updated record into the new file.
8. Stop the program.

PROGRAM:

```
#include <stdio.h>
#include <string.h>

struct teledir {
    int no;
    char name[30];
};

int main() {
    struct teledir t1, t2, t3;
    int i, n, p, newp;
    FILE *fp, *fp1;

    fp = fopen("td.txt", "w");

    printf("Enter the number of records: ");
    scanf("%d", &n);

    printf("Enter the records:\n");

    for (i = 0; i < n; i++) {
        scanf("%d %s", &t1.no, t1.name);
        fwrite(&t1, sizeof(struct teledir), 1, fp);
    }

    fclose(fp);

    fp = fopen("td.txt", "r");

    while (fread(&t2, sizeof(struct teledir), 1, fp) != 0) {
        printf("\t%d %s\n", t2.no, t2.name);
    }

    printf("Enter number to be modified and a new number: ");
    scanf("%d %d", &p, &newp);
    fclose(fp);
```

```

fp = fopen("td.txt", "r+");
fp1 = fopen("td1.txt", "w");

while (fread(&t2, sizeof(struct teledir), 1, fp) != 0) {

    if (t2.no == p) {
        fseek(fp, -sizeof(struct teledir), SEEK_CUR);
        t3.no = newp;
        strcpy(t3.name, t2.name);
        fwrite(&t3, sizeof(struct teledir), 1, fp1);
    }
    else {
        fwrite(&t2, sizeof(struct teledir), 1, fp1);
    }
}

fclose(fp);
fclose(fp1);

fp = fopen("td1.txt", "r");
while (fread(&t3, sizeof(struct teledir), 1, fp) != 0) {
    printf("\t%d\t%s\n", t3.no, t3.name);
}

fclose(fp);
return 0;
}

```

OUTPUT:

```

Enter the number of records: 3
Enter the records:
111 abc
222 xyz
333 nmo
    111 abc
    222 xyz
    333 nmo
Enter number to be modified and a new number: 222 9898
    111 abc
    9898 xyz
    333 nmo

```

RESULT:

The C program to update telephone details of an individual or a company into a telephone directory using a random access file has been executed successfully, and the output was verified.

AIM:

To write a C program to find the greatest of three numbers using the ternary operator with preprocessor directives.

ALGORITHM:

1. Start the program.
2. Define preprocessor directives for input.
3. Read three numbers from the user.
4. Use the ternary operator to determine the greatest number.
5. Print the greatest number.
6. Stop the program.

PROGRAM:

```
#include <stdio.h>

#define MAX(a, b) ((a) > (b) ? (a) : (b))

int main() {
    int num1, num2, num3, greatest;

    // Reading three numbers
    printf("Enter three numbers:\n");
    scanf("%d %d %d", &num1, &num2, &num3);

    // Using the ternary operator to find the greatest number
    greatest = MAX(num1, MAX(num2, num3));

    // Printing the result
    printf("The greatest number is: %d\n", greatest);

    return 0;
}
```

OUTPUT:

```
Enter three numbers:
10 25 15
The greatest number is: 25
```

RESULT:

This C program successfully finds the greatest of three numbers using the ternary operator defined through preprocessor directives.